

Studi'OS

Roadmap IA V2.1 - AI Library & Runtime Binding

V2.1 intègre le second audit de faisabilité. Décision de lancement : GO P0 uniquement. P1 ne doit pas commencer avant acceptation formelle du gate P0.

Pourquoi V2.1

La V2 a corrigé les blocages structurants de la V1. La V2.1 durcit P0 afin de trancher avant code les ambiguïtés restantes : sémantique exacte des scopes sans ACL projet parallèle, modèle de versioning, idempotence, audit/events, contrat CapabilityRequirement, évolution du manifeste DEC-0057, budget 256 Kio, confidentialité d'omitted[], fixtures neutres et comportement offline.

Invariant final

Même AgentDefinition + mêmes Rules + mêmes Skills + même contexte projet ; choix runtime/model différent par utilisateur. Studi'OS partage et résout ; le client local compose le Context Package ; le harness orchestre/exécute ; provider/model fournit l'intelligence.

P0 - Architecture Gate obligatoire

Pour l'agent d'implémentation : STOP avant tout code de production. Auditer les fichiers/DEC réels, produire les décisions ci-dessous et un plan P1. Une contradiction avec une DEC existante arrête le point concerné et doit être rapportée.

- Nommage : confirmer AgentDefinition (ou nom équivalent explicitement décidé) pour ne jamais confondre le rôle réutilisable avec Agent, identité/provenance opérationnelle existante. DEC obligatoire.
- Sémantique des scopes : définir précisément qui lit et écrit Studio, Project et User. Il n'existe pas d'ACL projet générale : ne pas en inventer une. Réutiliser Principal, ensure_can_write, ScopePolicy/deny-all et ownership. DEC obligatoire.
- Versioning : choisir explicitement le modèle compatible avec TECH/05 : lignes versionnées + pointeur actif, copy-on-write ou autre approche justifiée ; préciser locks, rollback, active version, immutable history et interaction avec version=409/append-only. DEC obligatoire.
- Idempotence : décider quelles créations/mutations de bibliothèque supportent Idempotency-Key conformément à TECH/02_API_CONTRACT et aux DEC d'idempotence existantes.
- Traçabilité : décider quelles mutations substantielles Rule/Skill/AgentDefinition/ModelProfile/Workflow produisent AIWorkLog/Event. Tout nouveau type d'événement suit contract-change ; ne pas ouvrir silencieusement TECH/03.
- CapabilityRequirement : définir le contrat abstrait neutre que le resolver P5 consommera et que le Runtime Registry P6 saura satisfaire, sans catalogue vendor.
- DEC-0057 / manifeste : statuer sur les kinds nécessaires (rule/skill/agentdefinition/profile ou autre), schema_version (reste 1 ou évolue), priorité dans le budget dur 256 Kio, et garantir qu'omitted[] ne contient jamais de chemin privé.
- Offline : la résolution pure fonctionne sans infra. Pour une lecture partagée indisponible, conserver server_unreachable -> omitted selon DEC-0057, sans outbox de lecture.
- Open strings : harness/provider/model restent des chaînes ouvertes ; aucun enum fermé, whitelist, ranking commercial ou nom de modèle utilisé comme rôle normatif.
- HTTP-first / MCP minimal : HTTP canonique, MCP subset additif et handlers minces. Concevoir les outils par use-case, pas un outil par entité.
- Schemas : stratégie de schémas Pydantic/outputSchema explicites avant extension contractuelle ; tenir compte de la dette dict[str,Any] et de DEC-0048.
- Secret boundary : credentials provider hors bibliothèque et hors surfaces partagées ; stockage/exécution locale. Aucun secret d'un utilisateur exposable à un autre.
- Context boundary : aucun compositeur Context Package VPS/MCP ; composition locale existante, manifeste éphémère, budget dur, frontière unidirectionnelle et absence de persistance serveur conservés.

Livrables obligatoires du gate P0

- DEC - AgentDefinition : nommage et séparation sémantique avec Agent provenance.
- DEC - AI Library scopes & authorization : lecture/écriture Studio/Project/User, mécanismes existants réutilisés, aucune ACL parallèle.
- DEC - AI Library versioning : stratégie retenue, locks, active version, rollback, immutable history, interaction version=409/append-only.
- Décision documentée - Idempotency-Key pour créations/mutations Library.
- Décision documentée - AIWorkLog/Event et éventuels nouveaux event types via contract-change.
- Contrat - CapabilityRequirement neutre pour P5/P6.

- Spécification compatibilité DEC-0057 : kinds, schema_version, ordre/priorité budget 256 Kio, omitted privacy, server_unreachable/offline.
- Stratégie HTTP-first/MCP minimal et stratégie Pydantic/outputSchema.
- Secret boundary explicite.
- Plan d'implémentation P1 court, fichiers/contrats affectés et tests attendus.

Gate P0 accepté seulement si tous ces livrables sont tranchés. Sinon : NO-GO P1.

Roadmap après acceptation de P0

P1 - Domaine canonique + sécurité + versioning

- Créer Rule, Skill, AgentDefinition, ModelProfile et le socle de Workflow déclaratif.
- Stable ID, nom, description, contenu/définition, scope, lifecycle, version, ownership, timestamps, dépendances.
- États au minimum draft/active/deprecated ; modèle de versioning décidé en P0 appliqué dès les migrations.
- Relations sans duplication : AgentDefinition->Rules/Skills/ModelProfile ; Skill->Rules ; Workflow->AgentDefinitions/Skills.
- Migrations Alembic réversibles ; auth via mécanismes existants ; aucun credential.
- Appliquer la décision de traçabilité P0 aux mutations substantielles.

Gate / sortie : Schéma, auth, versioning, audit et migrations validés.

P2 - Scopes et héritage

- Implémenter exactement la DEC de scopes P0 ; aucune ACL parallèle.
- Résolution déterministe Studio/Project/User et conflits explicitement définis.
- User privé par défaut selon la politique choisie ; accès Project/Studio selon les mécanismes existants.
- Tests ownership, isolation et refus d'accès.

Gate / sortie : Sémantique de scope prouvée par tests.

P3 - Model Profiles + CapabilityRequirement

- Représenter les exigences neutres : reasoning, coding, contexte, tools, multimodalité, local, coût, latence selon le contrat P0.
- Aucun ranking commercial, enum fermé ou whitelist.
- Les exemples normatifs utilisent provider_a/provider_b/model_a/model_b.

Gate / sortie : Profil et requirements vendor-neutral.

P4 - User Bindings

- Bindings ModelProfile->runtime/model et override AgentDefinition->runtime/model.
- Overrides projet et session temporaire ; priorité exacte documentée/testée.
- Auditer l'identité Machine.owner_user_id / JWT dashboard avant tout stockage.
- Persister uniquement les références/configurations non secrètes nécessaires.

Gate / sortie : Deux utilisateurs configurent différemment une définition commune.

P5 - Resolution Engine

- Resolver pur : AgentDefinition + dépendances + Rules + Skills + ModelProfile/CapabilityRequirement + bindings + overrides.
- Spécification harness-neutral + provenance complète (source, scope, version/lock, override).
- Erreur structurée si incompatibilité ; aucun fallback silencieux.
- Fonctionne sans infra pour la résolution pure ; aucune dépendance LLM.

Gate / sortie : MVP logique prouvé.

P6 - Runtime Registry

- Séparer Harness, Provider et Model ; chaînes ouvertes et capacités extensibles.
- Implémenter la satisfaction de CapabilityRequirement sans coupler le resolver à des catalogues vendor.
- Ollama/local est un runtime normal, pas un cas architectural séparé.

Gate / sortie : Compatibilité runtime vérifiable.

P7 - HTTP canonique

- Exposer les opérations nécessaires via services communs et Pydantic explicite.
- Appliquer la décision Idempotency-Key P0 aux créations/mutations concernées.
- Respecter auth, ownership, versioning, audit et contrats existants.
- Éviter les endpoints redondants.

Gate / sortie : HTTP = contrat de référence.

P8 - MCP minimal

- Subset HTTP additif, orienté use-cases, pas ~8 outils mécaniques.
- Handlers minces via mêmes services ; Principal avant idempotence lorsque requis.
- Respect DEC-0046/0048 et politique breaking/additive ; output schemas explicites.
- contract-change + contract-guardian pour toute surface touchée.

Gate / sortie : Surface MCP petite et stable.

P9 - Intégration Context Package locale

- Réutiliser providers 8.2, StudioApiClient et packages/studio-client/src/studio_client/context/.
- Appliquer exactement la décision P0 sur kinds/schema_version/priorité budget 256 Kio.
- omitted[] ne révèle jamais de chemin privé.
- server_unreachable -> omitted en lecture dégradée ; pas d'outbox de lecture.
- Aucun composeur VPS/MCP, second backend Bloc B ou persistance serveur du package/manifeste.

Gate / sortie : Bibliothèque intégrée au composeur local DEC-0057.

P10 - Adapters

- Claude Code puis OpenCode comme adapters locaux ; exemples produit non normatifs.
- Aucun if provider/model/harness dans le core.
- Une erreur adapter ne corrompt pas le canonique ; contrat générique reste référence.

Gate / sortie : Même définition traduite vers plusieurs harnesses.

P11 - Workflows déclaratifs

- Participants, ordre/dépendances, Rules/Skills, inputs/outputs.
- Exécution/orchestration côté harness ; toute orchestration serveur future nécessite une nouvelle DEC.

Gate / sortie : Workflow partageable, non exécuté par le VPS.

P12 - UI + Resolution Inspector

- Library : Rules, Skills, AgentDefinitions, Workflows, Model Profiles.
- Config user : runtimes, bindings, overrides ; config projet : ressources, locks, overrides.
- Inspector : version canonique, Rules/Skills, profile/requirements, runtime résolu, provenance et raisons.

Gate / sortie : Configuration visible et débogable.

P13 - Validation E2E

- Scopes, héritage, priorité, dépendances, locks, deprecated, missing deps, incompatibilités, isolation, permissions.
- HTTP/MCP schemas, auth par requête, audit/events, idempotence, Context Package local, offline dégradé, adapters.
- Fixture neutre : User A -> provider_a/model_a ; User B -> provider_b/model_b ; même AgentDefinition/Rules/Skills.
- studio-tester + offline-sync-testing lorsque pertinent ; migrations réversibles.

Gate / sortie : Sous-système accepté de bout en bout.

MVP

MVP cible : P0 -> P5 + chemin HTTP minimal nécessaire pour exercer le système. Il doit prouver définitions partagées, isolation, bindings personnels, CapabilityRequirement neutre, résolution déterministe, provenance, versioning/sécurité intégrés et absence de secrets. UI complète, adapters et workflows restent hors MVP.

Discipline d'implémentation

- Une phase à la fois : inspecter -> planifier -> petite tranche cohérente -> tests -> docs/DEC -> rapport -> commit indépendant si pertinent.
- contract-change + contract-guardian dès qu'un contrat est touché ; studio-tester pour validation ; offline-sync-testing sur les chemins concernés.
- HTTP reste canonique ; MCP reste un subset. Aucun backend parallèle Bloc B.
- Conserver l'offline/idempotence existants et les migrations réversibles.
- Ne pas pousser sans instruction explicite.
- Ne jamais résoudre une contradiction architecturale en inventant silencieusement une convention.

Instruction de démarrage

Exécuter P0 uniquement. Ne créer aucune migration, table, endpoint, outil MCP ou UI AI Library pendant P0. Retourner les DEC/décisions proposées, preuves repo, contradictions éventuelles et le plan P1. Attendre acceptation du gate avant P1.